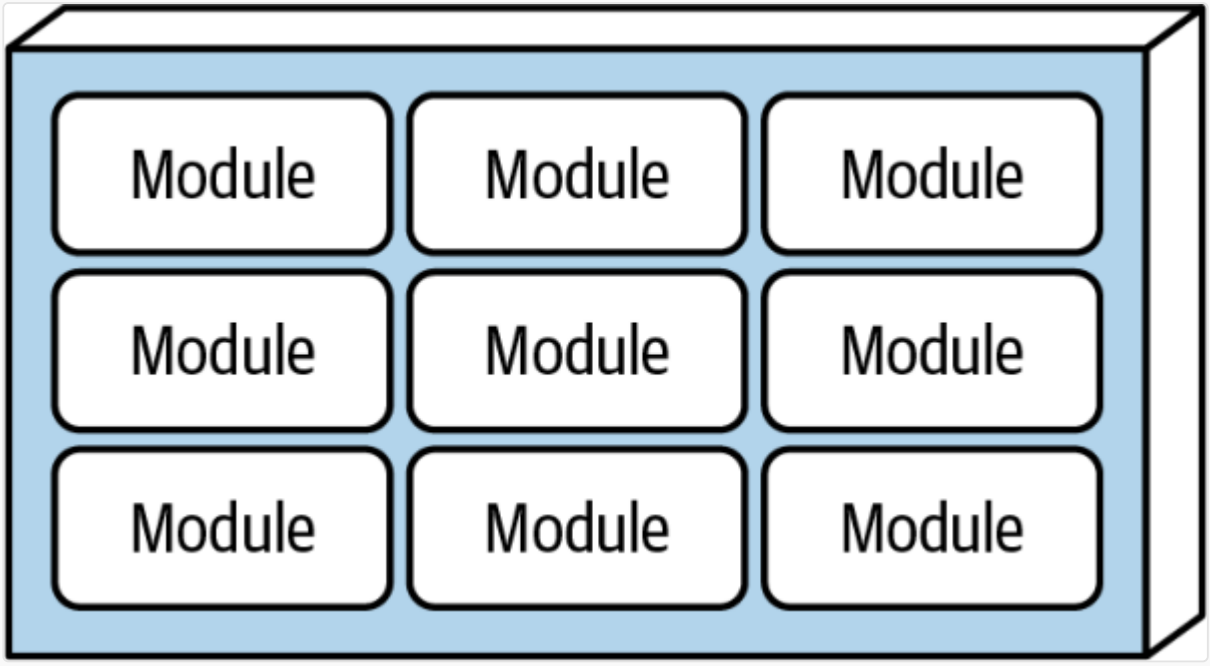
فصل ۱۱: سبک معماری تک‌پارچهٔ ماژولار

به لطفِ پذیرشِ گستردهٔ طراحی حوزه‌محور (DDD) و نیز توجهِ روزافزون به افراز حوزه‌ای، سبک معماری تک‌پارچهٔ ماژولار از زمانی که ویراست نخستِ این کتاب را در سالِ ۲۰۲۰ نوشتیم چنان محبوبیتی یافته است که تصمیم گرفتیم فصلی را در ویراست دوم به شرح (و رتبه‌بندیِ) آن اختصاص دهیم.

توپولوژی

چنان‌که از نامش پیداست، سبک معماری تک‌پارچهٔ ماژولار یک معماری تک‌پارچه است. از این رو، به‌صورتِ تنها یک واحدِ نرم‌افزاری مستقر می‌شود: یک فایلِ آرشیو وب (web archive یا WAR)، یک اسمبلی (assembly) واحد در .NET، یک فایلِ آرشیو سازمانی (enterprise archive یا EAR) در پلتفرم Java و مانندِ آن‌ها. از آنجا که تک‌پارچهٔ ماژولار معماری‌ای افرازشدهٔ حوزه‌ای به شمار می‌آید (یعنی معماری‌ای که بر پایهٔ حوزه‌های کسب‌وکار سازمان یافته است، نه قابلیت‌های فنی)، شکلِ هم‌ریخت (isomorphic shape) آن در قالبِ یک واحد استقرار تعریف می‌شود که کارکردهایش بر پایهٔ ناحیهٔ حوزه گروه‌بندی شده‌اند. شکل ۱۱-۱ توپولوژیِ معمولِ تک‌پارچهٔ ماژولار را نشان می‌دهد.

برای اینکه ماهیتِ حوزه‌محورِ تک‌پارچهٔ ماژولار را بهتر دریابید، معماری لایه‌ایِ سنتی (که در فصل ۱۰ شرح داده شد) را در نظر بگیرید. مؤلفه‌های آن بر پایهٔ قابلیت‌های فنی‌شان تعریف و سازمان‌دهی می‌شوند: لایه‌های Presentation، Business، Persistence و مانندِ آن‌ها. برای نمونه، منطقِ ارائه برای نگه‌داریِ اطلاعاتِ نمایهٔ مشتری ممکن است با مؤلفه‌ای با فضای نامِ com.app.presentation.customer.profile بازنمایی شود. گرهٔ سوم در این فضای نام نشان‌دهندهٔ دغدغهٔ فنیِ لایه است (در این مورد، لایهٔ Presentation).



شکل ۱۱-۱. در سبک معماری تک‌پارچهٔ ماژولار، کارکردها بر پایهٔ ناحیهٔ حوزه گروه‌بندی می‌شوند

در مقابل، مؤلفه‌های تک‌پارچهٔ ماژولار در درجهٔ نخست بر پایهٔ حوزه سازمان‌دهی می‌شوند. از این رو، در معماری تک‌پارچهٔ ماژولار، همان مؤلفهٔ نگه‌داریِ نمایهٔ مشتری با فضای نامِ `com.app.customer.profile` بازنمایی می‌شود. در اینجا گرهٔ سومِ فضای نام به یک دغدغهٔ حوزه‌ای اشاره دارد، نه یک دغدغهٔ فنی. بسته به پیچیدگیِ مؤلفه، فضای نام ممکن است پس از دغدغهٔ حوزه‌ای بر پایهٔ دغدغهٔ فنی نیز تقسیم شود؛ مانندِ `com.app.customer.profile.presentation` یا `com.app.customer.profile.business`.

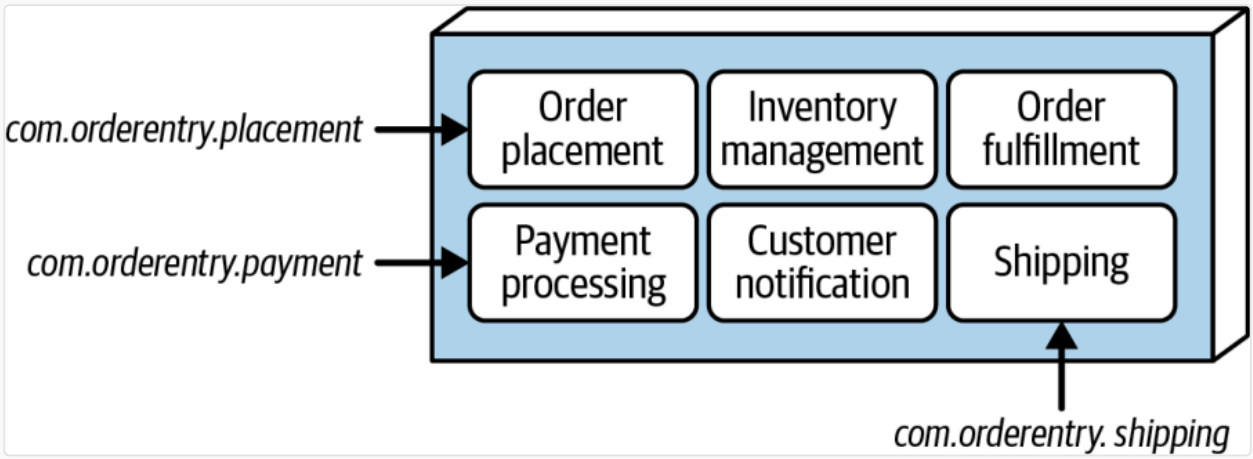
ویژگی‌های خاصِ سبک

در این سبک معماری، حوزه‌ها (یا در برخی موارد زیرحوزه‌ها) ماژول نامیده می‌شوند. ماژول‌ها را می‌توان به یکی از دو شیوه سازمان‌دهی کرد. ساده‌ترین معماری ساختارِ تک‌پارچه `(monolithic structure)` است که در آن همهٔ ماژول‌ها و مؤلفه‌های منطقیِ متناظرشان در یک پایگاه کدِ واحد جای دارند و با فضای نام یا ساختارِ دایرکتوری‌ای که در آن قرار گرفته‌اند از هم مرزبندی می‌شوند. گزینهٔ اندکی پیچیده‌تر ساختارِ ماژولار `(modular structure)` است که در آن هر ماژول به‌صورتِ یک مصنوعِ مستقل و خودبسنده `(self-contained artifact)`، مانندِ یک فایلِ `JAR` یا `DLL`، بازنمایی می‌شود و ماژول‌ها در هنگامِ استقرار در یک واحدِ نرم‌افزاریِ تک‌پارچه ترکیب می‌شوند.

مانندِ همه‌چیز در معماری نرم‌افزار، انتخاب میانِ این دو گزینهٔ ساختاری به عوامل و مصالحه‌های فراوانی بستگی دارد. بخش‌های زیر هر دو گزینه را معرفی می‌کنند و مصالحه‌های متناظرشان را بررسی می‌کنند.

ساختارِ تک‌پارچه

در ساختارِ تک‌پارچه، همهٔ ماژول‌هایی که سیستم را بازنمایی می‌کنند در یک مخزن کد منبعِ واحد جای دارند. همهٔ کدِ مرتبط با هر ماژول، هنگامِ تحویل یا انتشارِ نرم‌افزار، یک‌جا و به‌صورتِ یک واحد مستقر می‌شود. این گزینهٔ ساختاری در شکل ۱۱-۲ نشان داده شده است. هر ماژول با یک دایرکتوریِ سطح‌بالای جداگانه بازنمایی می‌شود که مؤلفه‌ها و هر زیرحوزه‌ای را که آن ماژول را می‌سازد در بر می‌گیرد.



شکل ۱۱-۲. نمونه‌ای از گزینهٔ ساختارِ تک‌پارچه

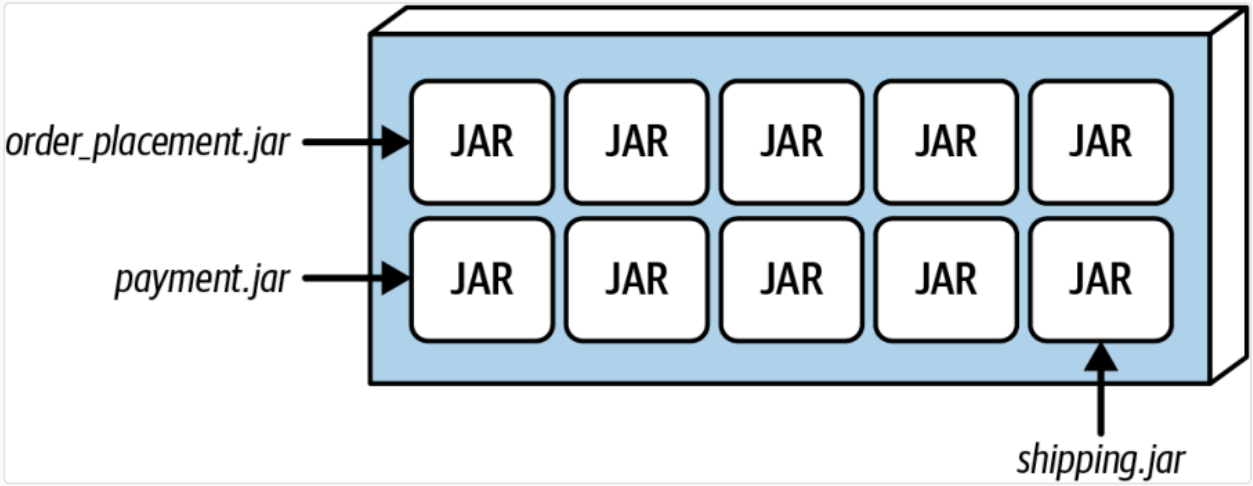
فضاهای نامِ زیر نشان می‌دهند که یک تک‌پارچهٔ ماژولار برای معماریِ نشان‌داده‌شده در شکل ۱۱-۲ ممکن است چه شکلی داشته باشد:

```
com.orderentry.orderplacement
com.orderentry.inventorymanagement
com.orderentry.paymentprocessing
com.orderentry.notification
com.orderentry.fulfillment
com.orderentry.shipping
```

این ساده‌ترین گزینه برای تک‌پارچهٔ ماژولار است: همهٔ کدِ منبعِ سیستم در یک جا قرار دارد و از این رو نگه‌داری، آزمون و استقرارش آسان‌تر است. اما برای حفظِ مرزهای هر ماژول به حاکمیتی سخت‌گیرانه نیاز است (بخش «حاکمیت» را در صفحهٔ ۱۷۲ ببینید). هرچند این گزینهٔ ساختاری ساده است، توسعه‌دهندگان تمایل دارند کدِ بیش از اندازه‌ای را میانِ ماژول‌ها بازاستفاده کنند و همچنین ارتباطاتِ بیش از حدی میانِ ماژول‌ها برقرار سازند (بخش «ارتباطِ میانِ ماژول‌ها» را در صفحهٔ ۱۶۸ ببینید). این رویه‌ها می‌توانند یک تک‌پارچهٔ ماژولارِ خوش‌ساخت را به یک توپِ بزرگِ گِلِ بی‌ساختار تبدیل کنند.

ساختارِ ماژولار

در ساختارِ ماژولار، ماژول‌ها به‌صورتِ مصنوع‌های خودبسنده (مانندِ فایل‌های `JAR` و `DLL`) بازنمایی می‌شوند و سپس در هنگامِ استقرار در قالبِ یک واحد استقرار کنارِ هم قرار می‌گیرند. شکل ۱۱-۳ این گزینه را با استفاده از فایل‌های `JAR` در پلتفرم `Java` نشان می‌دهد.



شکل ۱۱-۳. نمونه‌ای از گزینهٔ ساختارِ ماژولار با استفاده از فایل‌های `JAR`

مزیتِ این ساختار آن است که هر ماژول خودبسنده است و به تیم‌ها امکان می‌دهد روی ماژول‌های جداگانه کار کنند (بخش «ملاحظاتِ توپولوژی تیم» را در صفحهٔ ۱۷۴ ببینید)؛ بسیاری از اوقات حتی در مخزن کد منبعِ اختصاصیِ خودِ تیم برای آن ماژول‌ها. این گزینه زمانی خوب کار می‌کند که ماژول‌ها تا حدِ زیادی از ماژول‌های دیگر مستقل باشند. همچنین برای سیستم‌های بزرگ‌تر و پیچیده‌تری که هر ماژولشان به نوعِ متفاوتی از تخصص یا دانشِ کسب‌وکار نیاز دارد، به‌خوبی کار می‌کند. در گزینهٔ ساختارِ ماژولار، توسعه‌دهندگان کمتر به بازاستفادهٔ بیش از حدِ کد یا ارتباطِ بیش از حدِ ماژول‌ها با یکدیگر گرایش دارند (بخش «ارتباطِ میانِ ماژول‌ها» را در صفحهٔ ۱۶۸ ببینید). این گزینه همچنین معمولاً مرزهای تمیزتری میانِ ماژول‌ها و جداسازیِ کلیِ بهتری در دغدغه‌ها فراهم می‌آورد.

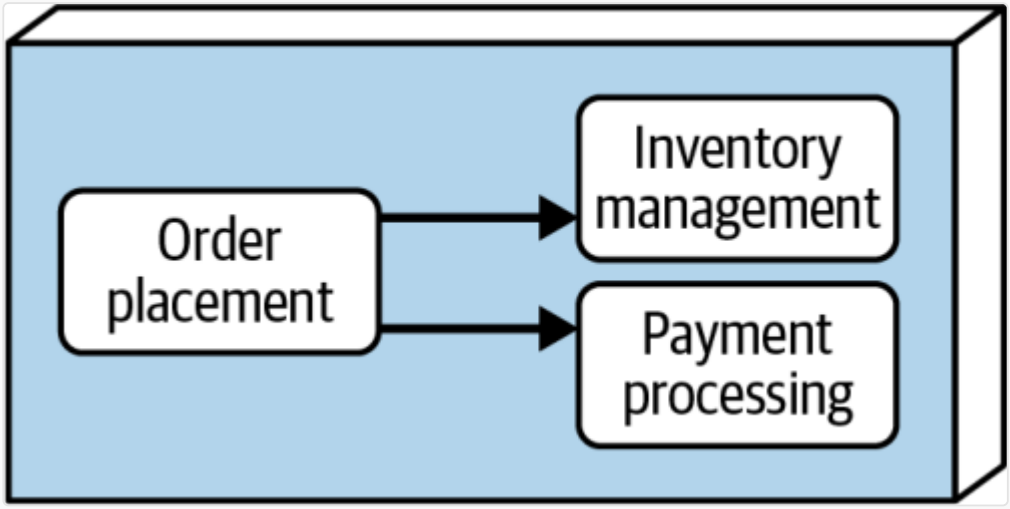
اما این گزینهٔ ساختاری زمانی اثربخشی‌اش را از دست می‌دهد که ماژول‌های وابسته به یکدیگر نیاز به ارتباط داشته باشند. در چنین وضعیتی، رویکردِ ساختارِ تک‌پارچه اثربخش‌تر است.

ارتباطِ میانِ ماژول‌ها

ارتباط میانِ ماژول‌ها در این سبک معماری هرگز چیزِ خوبی نیست، اما می‌پذیریم که در بسیاری از موارد ضروری است. برای نمونه، در معماریِ نشان‌داده‌شده در شکل ۱۱-۲، ماژولِ `OrderPlacement` باید با ماژولِ `InventoryManagement` ارتباط برقرار کند تا موجودیِ کالای سفارش‌داده‌شده را تنظیم کند و هر پردازشِ افزودهٔ دیگری را انجام دهد (برای نمونه، اگر موجودی بیش از حد کم باشد، سفارشِ کالای بیشتری بدهد). این ماژول همچنین باید با ماژولِ `PaymentProcessing` ارتباط برقرار کند تا پرداختِ یک سفارش را اعمال کند. دو گزینهٔ اصلی برای ارتباط میانِ ماژول‌ها وجود دارد که در بخش‌های زیر شرح می‌دهیم.

رویکردِ همتابه‌همتا

سرراست‌ترین راه‌حل، ارتباطِ سادهٔ همتابه‌همتا (peer-to-peer) میانِ ماژول‌هاست. در این رویکرد، یک فایلِ کلاس در یک ماژول، کلاسی را در ماژولی دیگر نمونه‌سازی می‌کند و متد یا متدهای لازمِ آن کلاس را برای انجامِ عملیات فرامی‌خواند (شکل ۱۱-۴ را ببینید).



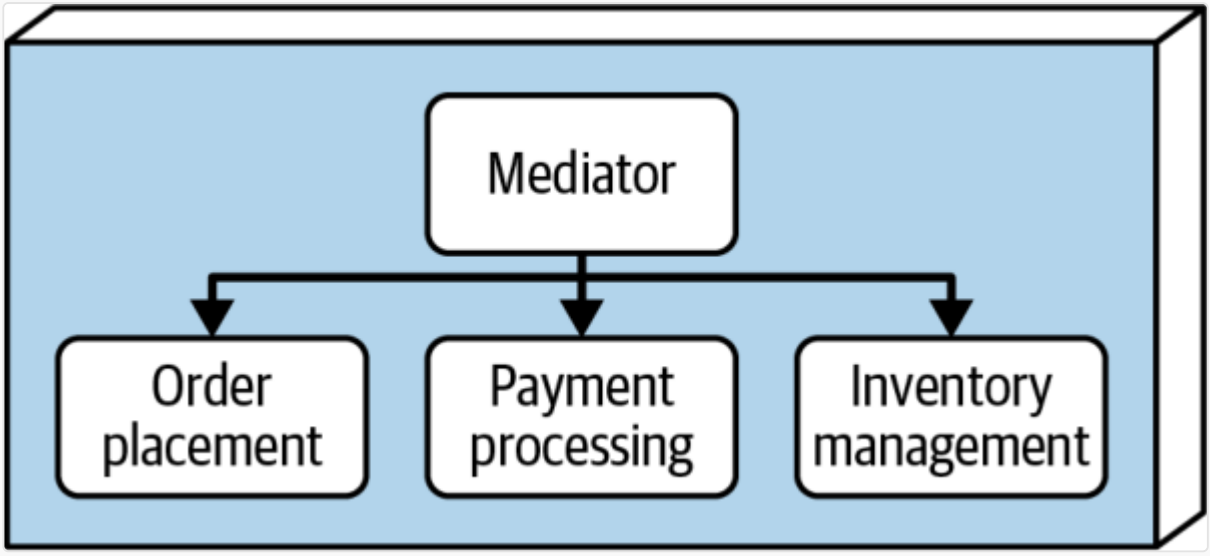
شکل ۱۱-۴. ارتباطِ همتابه‌همتا میانِ ماژول‌ها

یکی از مشکلاتِ رویکردِ تک‌پارچه آن است که نمونه‌سازیِ هر کلاسی از ماژولی دیگر برای توسعه‌دهندگان بیش از حد آسان است. همین امر لغزیدن از یک معماریِ خوش‌ساختار به ضدالگوی توپِ بزرگِ گِل را آسان می‌کند (شکل ۹-۱ را ببینید).

اما در ساختارِ ماژولار، کلاس‌های ماژولی دیگر ممکن است به جای یک دایرکتوریِ جداگانه در مخزن کد منبع، در مصنوع‌های بیرونیِ جداگانه (فایل‌های JAR یا DLL) قرار داشته باشند. ماژولی که با ماژول‌های دیگر ارتباط دارد، بدونِ ارجاع به کلاس‌ها کامپایل نمی‌شود؛ یعنی توسعه‌دهنده باید میانِ آن ماژول‌ها یک وابستگیِ زمانِ کامپایل (compile-time dependency) پدید آورد. پاسخِ معمول به این مشکل ساختنِ یک کلاسِ واسطِ مشترک (shared interface class) میانِ آن ماژول‌هاست (در یک فایلِ JAR یا DLL مشترکِ جداگانه)، تا هر ماژول بتواند مستقل از ماژول‌های دیگر کامپایل شود. در هر حال، ارتباطِ بیش از حد میانِ ماژول‌ها در رویکردِ ساختارِ ماژولار به ضدالگوی جهنمِ DLL (DLL Hell) یا جهنمِ JAR (JAR Hell)، به ضدالگوی جهنمِ Java می‌انجامد (یا در پلتفرم).

رویکردِ میانجی

رویکردِ میانجی (mediator) با به‌کارگیریِ یک مؤلفهٔ میانجی که لایهٔ انتزاعی (abstraction layer) میانِ ماژول‌ها پدید می‌آورد، ماژول‌ها را از هم جدا می‌کند. میانجی نقشِ هماهنگ‌کننده را دارد: درخواست‌ها را می‌پذیرد و آن‌ها را به ماژول‌های مناسب می‌رساند. شکل ۱۱-۵ این رویکرد را نشان می‌دهد.

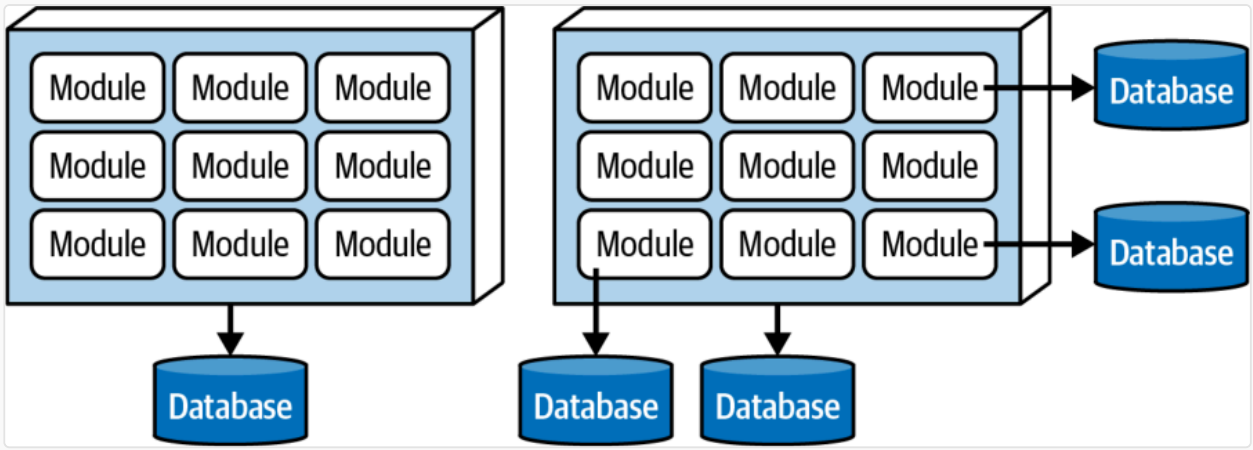


شکل ۱۱-۵. میانجی ماژول‌ها را از هم جدا می‌کند تا ناچار به ارتباط با یکدیگر نباشند

خوانندهٔ تیزبین درمی‌یابد که هرچند رویکردِ میانجی ماژول‌ها را از هم جدا می‌کند، هر ماژول در عمل به میانجی کوپل شده است. این رویکرد همهٔ کوپلینگ و وابستگی‌ها را از میان نمی‌برد، اما معماری را ساده می‌کند و ماژول‌ها را از یکدیگر مستقل نگه می‌دارد. توجه کنید که این میانجی است، نه ماژول‌های وابسته، که برای فراخوانیِ کارکردهای ماژول‌های دیگر به نوعی `API` یا واسط نیاز دارد.

توپولوژی‌های داده

از آنجا که معماری تک‌پارچهٔ ماژولار معمولاً به‌صورتِ تنها یک واحدِ نرم‌افزاری مستقر می‌شود، به‌طورِ معمول بر توپولوژیِ پایگاه‌دادهٔ تک‌پارچه `(monolithic database topology)` تکیه دارد. به‌کارگیریِ یک پایگاه‌دادهٔ واحد به کاهشِ ارتباط میانِ ماژول‌ها کمک می‌کند، زیرا داده‌ها مشترک‌اند. اما اگر ماژول‌ها از یکدیگر مستقل باشند و کارکردهای مشخصی انجام دهند، می‌توانند پایگاه‌داده‌های خودشان را نیز داشته باشند که دادهٔ زمینه‌ایِ `(contextual data)` مشخصی را در خود جای می‌دهند؛ حتی با اینکه خودِ معماری تک‌پارچه است. شکل ۱۱-۶ این دو گزینهٔ توپولوژیِ پایگاه‌داده را نشان می‌دهد.



شکل ۱۱-۶. داده‌ها می‌توانند تک‌پارچه باشند یا هر ماژول می‌تواند پایگاه‌دادهٔ خودش را داشته باشد

ملاحظات ابری

هرچند معماری‌های تک‌پارچهٔ ماژولار را می‌توان در محیط‌های ابری مستقر کرد (به‌ویژه اگر سیستم کوچک باشد)، این معماری‌ها به‌طورِ کلی برای استقرار در ابر چندان مناسب نیستند: ماهیتِ تک‌پارچه‌شان احتمالِ بهره‌گیری از تأمینِ منابع بر حسبِ تقاضا `(on-demand provisioning)` را که محیط‌های ابری عرضه می‌کنند، کاهش می‌دهد. با این حال، سیستم‌های کوچک‌تری که با این سبک معماری پیاده‌سازی شده‌اند، همچنان می‌توانند از بسیاری از سرویس‌های ابری، مانندِ ذخیره‌سازیِ فایل، پایگاه‌داده و پیام‌رسانی، بهره ببرند.

ریسک‌های رایج

مانندِ هر سیستمِ تک‌پارچهٔ دیگری، ریسکِ اصلیِ سبک معماری تک‌پارچهٔ ماژولار آن است که ممکن است چنان بزرگ شود که نتوان آن را به‌درستی نگه‌داری، آزمون و مستقر کرد. معماری‌های تک‌پارچه به‌خودیِ خود بد نیستند؛ مشکلات زمانی آغاز می‌شوند که بیش از حد بزرگ شوند. معنای «بیش از حد بزرگ» از سیستمی به سیستمِ دیگر فرق می‌کند، اما برخی از نشانه‌های هشداردهنده‌ای که از بزرگیِ بیش از حدِ سیستم خبر می‌دهند از این قرارند:

- اعمالِ تغییرات بیش از حد طول می‌کشد.
- وقتی بخشی از سیستم تغییر می‌کند، بخش‌های دیگر به‌طورِ غیرمنتظره از کار می‌افتند.
- اعضای تیم هنگامِ اعمالِ تغییرات سرِ راهِ یکدیگر قرار می‌گیرند.
- راه‌اندازیِ سیستم بیش از حد طول می‌کشد.

ریسکِ دیگر، زیاده‌روی در بازاستفادهٔ کد است. بازاستفاده و اشتراکِ کد بخشی ضروری از توسعهٔ نرم‌افزار است، اما در این سبک معماری، بازاستفادهٔ بیش از حدِ کد مرزهای ماژول‌ها را محو می‌کند و معماری را به قلمروِ پرخطرِ تک‌پارچهٔ بی‌ساختار `(unstructured monolith)` می‌کشاند: معماری‌ای تک‌پارچه با کدی چنان درهم‌تنیده که دیگر نمی‌توان

گره‌اش را گشود.

ارتباطِ بیش از حد میانِ ماژول‌ها `(intermodule communication)` ریسکِ دیگری در این سبک معماری است. در حالتِ آرمانی، ماژول‌ها باید مستقل و خودبسنده باشند. چنان‌که اشاره کردیم، ارتباطِ برخی ماژول‌ها با ماژول‌های دیگر، به‌ویژه در یک گردش‌کارِ پیچیده، عادی (و گاهی ضروری) است. اما اگر ارتباطِ متقابلِ میانِ ماژول‌ها بیش از حد باشد، نشانهٔ روشنی است که حوزه‌ها احتمالاً از همان ابتدا به‌درستی تعریف نشده‌اند. در چنین مواردی، ارزش دارد زمان و تأملِ بیشتری صرفِ بازتعریفِ حوزه‌ها کنید تا گردش‌کارهای پیچیده و وابستگی‌های متقابل را در خود جای دهند.

حاکمیت

مصنوعِ اصلی در سبک تک‌پارچهٔ ماژولار، ماژول است؛ ماژول نمایانگرِ یک حوزه یا زیرحوزهٔ مشخص است و معمولاً با ساختارِ دایرکتوری یا فضای نام (یا ساختارِ بسته‌ها `(package structure)` در پلتفرم `Java`) بازنمایی می‌شود. بنابراین، یکی از نخستین شکل‌های حاکمیت خودکار که معماران می‌توانند به کار گیرند، تعریفِ ماژول‌های به‌کاررفته در معماری و تضمینِ انطباق با آن‌هاست.

معماران برای نوشتنِ وارسی‌های حاکمیتِ خودکار از انبوهی از ابزارها بهره می‌گیرند، از جمله `ArchUnit` برای پلتفرم `Java`، `ArchUnitNet` و `NetArchTest` برای پلتفرم `.NET`، `PyTestArch` برای `Python` و `TSArch` برای `TypeScript` و `JavaScript`. شبه‌کدِ `(pseudocode)` مثال ۱۱-۱ تضمین می‌کند که همهٔ کدِ منبعِ بازنمایی‌شده در نمونهٔ معماریِ شکل ۱۱-۲ زیرِ یکی از فضاهای نامِ فهرست‌شده قرار گیرد؛ فضاهای نامی که هر یک نمایانگرِ یکی از ماژول‌های تعریف‌شدهٔ سیستم‌اند.

مثال ۱۱-۱. شبه‌کد برای تضمینِ پیرویِ کد از ماژول‌های تعریف‌شدهٔ سیستم

```
# The following namespaces represent the modules in the system
LIST module_list = {
   com.orderentry.orderplacement,
   com.orderentry.inventorymanagement,
   com.orderentry.paymentprocessing,
   com.orderentry.notification,
   com.orderentry.fulfillment,
   com.orderentry.shipping
   }

# Get the list of namespaces in the system
LIST namespace_list = get_all_namespaces(root_directory)

# Make sure all the namespaces start with one of the listed modules
FOREACH namespace IN namespace_list {
   IF NOT namespace.starts_with(module_list) {
      send_alert(namespace)
   }
}
```

اگر توسعه‌دهنده‌ای هرگونه فضای نام یا دایرکتوریِ سطح‌بالای دیگری بیرون از ماژول‌های تعریف‌شده بسازد، هشداری دریافت می‌کند که نشان می‌دهد کدِ منبع با معماری منطبق نیست.

این شکل از حاکمیت با گزینهٔ ساختارِ تک‌پارچهٔ این سبک معماری به‌خوبی کار می‌کند (بخش «ساختارِ تک‌پارچه» را در صفحهٔ ۱۶۷ ببینید)، اما با گزینهٔ ساختارِ ماژولار (بخش «ساختارِ ماژولار» را در صفحهٔ ۱۶۸ ببینید) چالش‌برانگیز است، زیرا ممکن است کد در یک مخزن کد منبعِ تک‌پارچهٔ واحد قرار نداشته باشد. در گزینهٔ ساختارِ ماژولار، هر ماژول باید جداگانه آزموده شود؛ چنان‌که در مثال ۱۱-۲ نشان داده شده است.

مثال ۱۱-۲. شبه‌کد برای اعتبارسنجیِ ماژولِ `InventoryManagement`

```
# Get the list of namespaces in the system
LIST namespace_list = get_all_namespaces(root_directory)

# Make sure all the namespaces start with com.orderentry.inventorymanagement
FOREACH namespace IN namespace_list {
   IF NOT namepace.starts_with("com.orderentry.inventorymanagement") {
      send_alert(namespace)
   }
}
```

راهِ دیگرِ حاکمیت بر معماری تک‌پارچهٔ ماژولار، کنترلِ میزانِ ارتباط میانِ ماژول‌هاست. تعریفِ ارتباطِ «بیش از حد» بسیار ذهنی است و از سیستمی به سیستمِ دیگر فرق می‌کند، اما در بیشترِ موارد معماران باید بکوشند شمارِ وابستگی‌های متقابلِ میانِ ماژول‌ها را به کمترین حد برسانند. مثال ۱۱-۳ شبه‌کدی را نشان می‌دهد که تضمین می‌کند بیشینهٔ کلِ وابستگی‌های متقابل از حدِ پنج نقطهٔ ارتباط (یا نقطهٔ کوپلینگ) فراتر نرود.

مثال ۱۱-۳. شبه‌کد برای محدود کردنِ شمارِ کلِ وابستگی‌های هر ماژول

```
# Walk the directory structure, gathering modules and the source code files
# contained within those modules
LIST module_list = {
   com.orderentry.orderplacement,
   com.orderentry.inventorymanagement,
   com.orderentry.paymentprocessing,
   com.orderentry.notification,
   com.orderentry.fulfillment,
   com.orderentry.shipping
   }
MAP module_source_file_map
FOREACH module IN module_list {
  LIST source_file_list = get_source_files(module)
  ADD module, source_file_list TO module_source_file_map
}

# Determine how many references exist for each source file and send an alert if
# the system's total dependency count is greater than 5
FOREACH module, source_file_list IN module_source_file_map {
  FOREACH source_file IN source_file_list {
    incoming count = used_by_other_module(source_file, module_source_file_map) {
    outgoing_count = uses_other_module(source_file) {
    total_count = incoming count + outgoing count
  }
  IF total_count > 5 {
    send_alert(module, total_count)
  }
}
```

آخرین شکلِ حاکمیتِ خودکار، تضمینِ استقلالِ ماژول‌ها از یکدیگر با منعِ ارتباطِ یک ماژولِ مشخص با ماژولی دیگر است. برای نمونه، در شکل ۱۱-۲، ماژولِ `OrderPlacement` نباید با ماژولِ `Shipping` ارتباط برقرار کند. مثال ۱۱-۴ کدِ `ArchUnit` به زبانِ `Java` را برای حاکمیت بر این وابستگی نشان می‌دهد.

مثال ۱۱-۴. کدِ `ArchUnit` برای حاکمیت بر محدودیت‌های وابستگی میانِ ماژول‌های مشخص

```
public void order_placement_cannot_access_shipping() {
   noClasses().that()
   .resideInAPackage("..com.orderentry.orderplacement..")
   .should().accessClassesThat()
   .resideInAPackage("..com.orderentry.shipping..")
   .check(myClasses);
}
```

ملاحظاتِ توپولوژی تیم

از آنجا که تک‌پارچهٔ ماژولار معماری‌ای افرازشدهٔ حوزه‌ای به شمار می‌آید، بهترین کارکرد را زمانی دارد که تیم‌ها نیز بر پایهٔ ناحیهٔ حوزه هم‌راستا شده باشند (مانندِ تیم‌های چندتخصصی با تخصص‌یابی). وقتی نیازمندی‌ای حوزه‌محور از راه می‌رسد، یک تیمِ چندتخصصیِ متمرکز بر حوزه می‌تواند با هم روی آن قابلیت کار کند؛ از منطقِ ارائه گرفته تا پایگاه‌داده.

در مقابل، تیم‌هایی که بر پایهٔ دسته‌بندی‌های فنی سازمان یافته‌اند (مانندِ تیم‌های رابط کاربری، تیم‌های سمت سرور، تیم‌های پایگاه‌داده و مانندِ آن‌ها)، عمدتاً به دلیلِ افراز حوزه‌ایِ این سبک معماری، با آن به‌خوبی کار نمی‌کنند. سازمان‌دهیِ نیازمندی‌های حوزه‌محور میانِ تیم‌هایی که بر پایهٔ مهارت‌های فنی شکل گرفته‌اند، نیازمندِ هماهنگی و ارتباطاتِ گسترده‌ای است که اغلب دشوار از آب درمی‌آید.

در ادامه چند ملاحظه برای هم‌راستاسازیِ توپولوژی‌های تیمِ مشخصی که در بخش «توپولوژی‌های تیم و معماری» در صفحهٔ ۱۵۱ معرفی شدند، با سبک تک‌پارچهٔ ماژولار آمده است:

تیم‌های هم‌راستا با جریان: تیم‌های هم‌راستا با جریان معمولاً مالکِ جریان در سیستم از آغاز تا پایان‌اند و این با شکلِ تک‌پارچه و به‌طورِ کلی خودبسندهٔ تک‌پارچهٔ ماژولار به‌خوبی جور درمی‌آید.

تیم‌های توانمندساز: به دلیلِ سطحِ بالای ماژولاریتی و جداسازی دغدغه‌ها در این سبک، توپولوژی‌های تیمِ توانمندساز نیز به‌خوبی کار می‌کنند. متخصصان و اعضای میان‌بُرشی تیم می‌توانند با افزودنِ ماژول‌های تازه به سیستم، پیشنهادهایی بدهند و آزمایش‌هایی انجام دهند، با کمترین اثر بر ماژول‌های موجودِ دیگر.

تیم‌های زیرسیستم پیچیده: هر ماژول در معماری تک‌پارچهٔ ماژولار معمولاً بر پایهٔ حوزه یا زیرحوزه‌اش نقشِ مشخصی ایفا می‌کند (مانندِ `PaymentProcessing`). این ویژگی با توپولوژیِ تیمِ زیرسیستم پیچیده به‌خوبی کار می‌کند، زیرا اعضای مختلفِ تیم می‌توانند مستقل از دیگر اعضای تیم (و دیگر ماژول‌ها) بر پردازشِ پیچیدهٔ یک حوزه یا زیرحوزه تمرکز کنند.

تیم‌های پلتفرم: توسعه‌دهندگان می‌توانند با به‌کارگیریِ ابزارها، سرویس‌ها، `API`ها و کارهای مشترک از مزایای توپولوژیِ تیم‌های پلتفرم بهره ببرند؛ این امر عمدتاً به لطفِ درجهٔ بالای ماژولاریتی در این سبک معماری ممکن است.

ویژگی‌های سبک

در جدول رتبه‌بندی ویژگی‌ها (شکل ۱۱-۷)، رتبهٔ یک‌ستاره یعنی آن ویژگی معماریِ مشخص در این معماری به‌خوبی پشتیبانی نمی‌شود، در حالی که رتبهٔ پنج‌ستاره یعنی آن ویژگی معماری یکی از قوی‌ترین جنبه‌های این سبک معماری است. ویژگی‌هایی که در این کارنامه آمده‌اند در فصل ۴ شرح و تعریف شده‌اند.

سبک معماری تک‌پارچهٔ ماژولار معماری‌ای افرازشدهٔ حوزه‌ای است، زیرا منطقِ برنامه در آن به ماژول‌ها افراز می‌شود. از آنجا که این سبک معمولاً به‌صورتِ استقرارِ تک‌پارچه پیاده‌سازی می‌شود، کوانتوم معماری‌اش به‌طورِ معمول ۱ است.

هزینهٔ کلی، سادگی و ماژولاریتی نقاطِ قوتِ اصلیِ سبک معماری تک‌پارچهٔ ماژولارند. این معماری‌ها، از آنجا که ذاتاً تک‌پارچه‌اند، پیچیدگی‌های سبک‌های معماری توزیع‌شده را ندارند. ساده‌ترند، فهمشان آسان‌تر است و ساخت و نگه‌داری‌شان نسبتاً کم‌هزینه است. ماژولاریتیِ معماری از راهِ جداسازی دغدغه‌ها میانِ ماژول‌های گوناگون، که نمایانگرِ حوزه‌ها و زیرحوزه‌ها هستند، به دست می‌آید.

استقرارپذیری و آزمون‌پذیری، هرچند فقط دوستاره‌اند، به دلیلِ سطحِ ماژولاریتیِ تک‌پارچهٔ ماژولار، اندکی بالاتر از معماری لایه‌ای رتبه می‌گیرند. با این حال، این سبک معماری همچنان یک تک‌پارچه است: از این رو، تشریفات، ریسک، بسامدِ استقرار و کامل‌بودنِ آزمون بر این امتیازها اثرِ منفی می‌گذارند.

کشسانی و مقیاس‌پذیری در معماری تک‌پارچهٔ ماژولار رتبهٔ بسیار پایینی (یک‌ستاره) دارند که دلیلِ اصلی‌اش استقرارهای تک‌پارچه است. هرچند می‌توان برخی کارکردهای درونِ یک تک‌پارچه را بیش از کارکردهای دیگر مقیاس‌پذیر کرد، این تلاش معمولاً به فنونِ طراحیِ بسیار پیچیده‌ای نیاز دارد (مانندِ چندریسمانی، پیام‌رسانی داخلی و دیگر رویه‌های پردازش موازی) که این معماری برایشان مناسب نیست.

	Architectural characteristic	Star rating
	Overall cost	$
Structural	Partitioning type	Domain
	Number of quanta	1
	Simplicity	★★★★★
	Modularity	★★
Engineering	Maintainability	★★
	Testability	★★
	Deployability	★★
	Evolvability	★★
Operational	Responsiveness	★★★
	Scalability	★
	Elasticity	★
	Fault tolerance	★

شکل ۱۱-۷. رتبه‌بندیِ ستاره‌ایِ ویژگی‌های معماری برای تک‌پارچهٔ ماژولار

استقرارهای تک‌پارچهٔ معماری تک‌پارچهٔ ماژولار از تحمل خطا پشتیبانی نمی‌کنند. اگر بخشِ کوچکی از معماری دچارِ خطای کمبود حافظه شود، کلِ واحدِ برنامه از کار می‌افتد. افزون بر این، مانندِ بیشترِ برنامه‌های تک‌پارچه، در دسترس‌بودنِ کلی از میانگین زمانِ بازیابیِ بالا آسیب می‌بیند؛ زمانِ راه‌اندازی معمولاً بر حسب دقیقه سنجیده می‌شود.

چه زمانی استفاده کنیم

سبک معماری تک‌پارچهٔ ماژولار، به دلیلِ سادگی و هزینهٔ پایینش، گزینهٔ مناسبی برای مواجهه با محدودیت‌های شدیدِ بودجه و زمان است. این سبک همچنین گزینهٔ خوبی برای آغازِ کار با یک سیستمِ جدید است. اگر جهت‌گیریِ معماریِ `(architectural direction)` سیستم هنوز روشن نیست، اغلب اثربخش‌تر است که با یک تک‌پارچهٔ ماژولار آغاز کنید و بعدها به سبک معماریِ توزیع‌شدهٔ پیچیده‌تر و پرهزینه‌تری، مانندِ معماری مبتنی بر سرویس (فصل ۱۴ را ببینید) یا میکروسرویس‌ها (فصل ۱۸ را ببینید)، کوچ کنید؛ تا اینکه یکراست به سراغِ معماری توزیع‌شده بروید.

تک‌پارچهٔ ماژولار همچنین گزینهٔ خوبی برای تیم‌های حوزه‌محور `(domain-focused teams)`، مانندِ تیم‌های چندتخصصی با تخصص‌یابی، است. این امر به هر تیم امکان می‌دهد از آغاز تا پایان بر ماژولِ مشخصی در معماری تمرکز کند و کمترین هماهنگی را با دیگر تیم‌های حوزه‌ای داشته باشد. این سبک معماری همچنین برای موقعیت‌هایی که بیشترِ تغییرات در سیستم حوزه‌محورند (مانندِ افزودنِ تاریخِ انقضا به اقلامِ فهرستِ علاقه‌مندی‌های `(wishlist)` یک مشتری) بسیار مناسب است.

سرانجام، از آنجا که تک‌پارچهٔ ماژولار معماری‌ای افرازشدهٔ حوزه‌ای است، برای تیم‌هایی که `DDD` را به کار می‌بندند بسیار مناسب است.

چه زمانی استفاده نکنیم

دلیلِ اصلیِ استفاده نکردن از این سبک معماری، زمانی است که سیستم‌ها یا محصولات به سطوحِ بالایی از برخی ویژگی‌های عملیاتی، مانندِ مقیاس‌پذیری، کشسانی، در دسترس‌بودن، تحمل خطا، پاسخ‌گویی و کارایی، نیاز دارند. مانندِ بیشترِ معماری‌های تک‌پارچه، تک‌پارچهٔ ماژولار برای این دغدغه‌های معماری مناسب نیست.

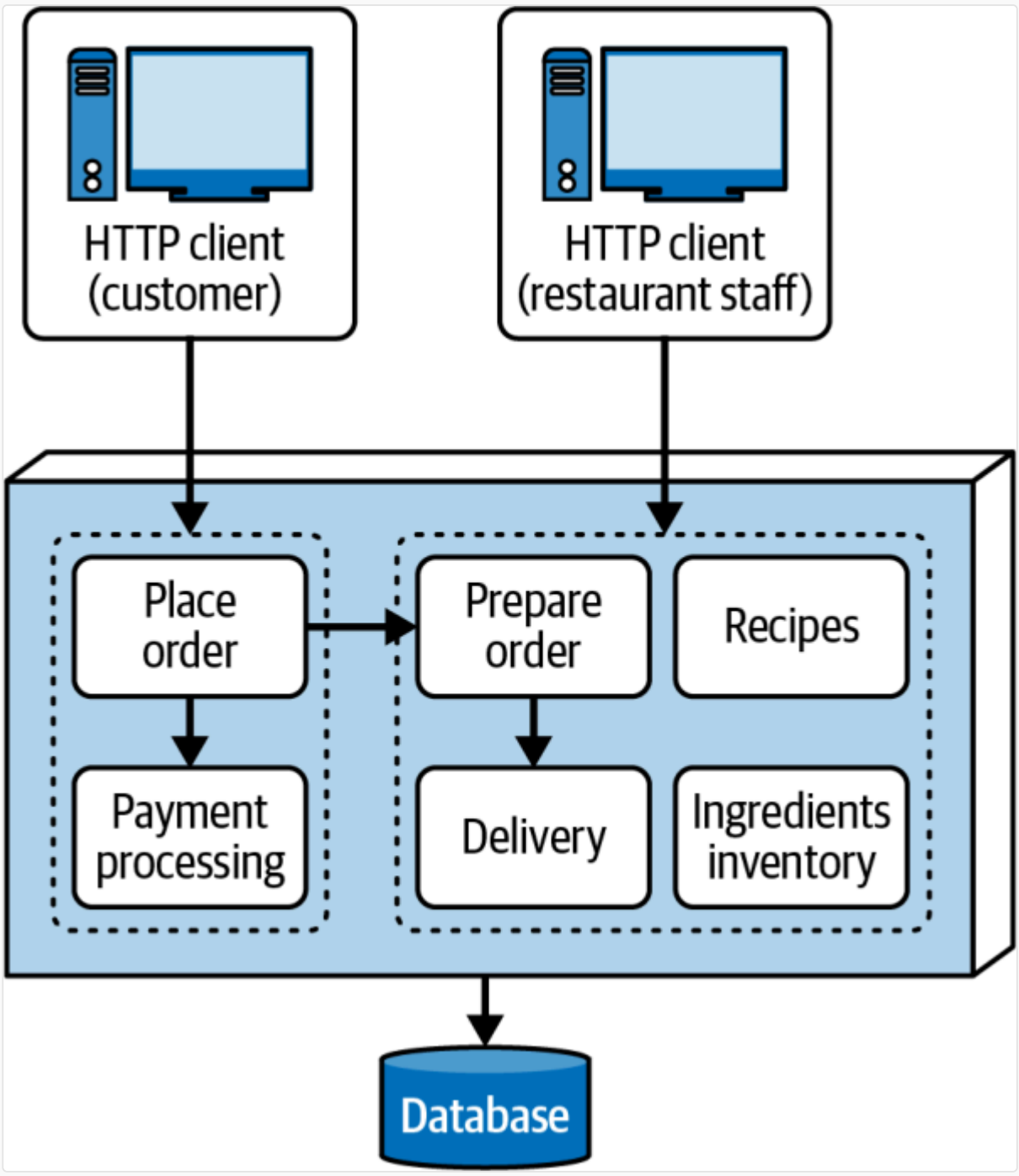
وقتی بیشترِ تغییرات ماهیتِ فنی دارند، مانندِ جایگزینیِ پیوستهٔ رابط کاربری یا فناوریِ پایگاه‌داده، از به‌کارگیریِ تک‌پارچهٔ ماژولار پرهیز کنید. از آنجا که این معماری به‌صورتِ حوزه‌ای افراز شده است، چنین تغییراتی بر همهٔ ماژول‌ها اثر می‌گذارند و معمولاً به ارتباط و هماهنگیِ چشمگیری میانِ تیم‌های حوزه‌ای نیاز دارند. در این موقعیت‌ها، سبک معماری لایه‌ای (فصل ۱۰ را ببینید) گزینهٔ بسیار بهتری است.

نمونه‌ها و موارد کاربرد

`EasyMeals` رستورانِ محلیِ تازه‌کاری است که غذا را تحویلِ درِ منزل می‌دهد و مشتریانش افرادِ شاغلی‌اند که پس از یک روزِ پرمشغله، همیشه وقتِ پختنِ غذا در خانه را ندارند. مشتریانِ گرسنه می‌توانند یک شامِ خوب را برخط سفارش دهند و آن را ظرفِ یک ساعت درِ خانه‌شان تحویل بگیرند.

این رستوران، به‌عنوانِ یک رستورانِ کوچک و محلی، نیازِ چندانی به مقیاس‌پذیری یا پاسخ‌گوییِ بالا ندارد. و چون بودجه‌اش محدود است، نمی‌خواهد پولِ زیادی صرفِ یک سیستمِ نرم‌افزاریِ پرزرق‌وبرق کند. شکلِ این مسئلهٔ کسب‌وکاری، تک‌پارچهٔ ماژولار را به گزینهٔ خوبی برای `EasyMeals` تبدیل می‌کند.

شکل ۱۱-۸ نشان می‌دهد که سیستمِ سادهٔ مدیریتِ رستورانِ `EasyMeals` با سبک معماری تک‌پارچهٔ ماژولار ممکن است چه شکلی داشته باشد.



شکل ۱۱-۸. یک سیستمِ کوچکِ سفارش‌گیری و مدیریتِ رستوران با سبک تک‌پارچهٔ ماژولار

مشتریان از راهِ یک رابط کاربریِ اختصاصی و متفاوت به ماژول‌های `PlaceOrder` و `PaymentProcessing` دسترسی دارند. فضاهای نامِ زیر نمایانگرِ هر یک از ماژول‌های این سیستم‌اند:

```
com.easymeals.placeorder
com.easymeals.payment
com.easymeals.prepareorder
com.easymeals.delivery
com.easymeals.recipes
com.easymeals.inventory
```

ماژولِ `PlaceOrder` به هر مشتری امکان می‌دهد منو را ببیند، اقلامی را برگزیند، نام، نشانی و اطلاعاتِ پرداختش را وارد کند و سفارش را ثبت کند. مؤلفه‌های این ماژول را می‌توان با فضاهای نامِ زیر بازنمایی کرد که کدِ منبعِ پیاده‌سازِ هر یک از این کارکردهای اصلی را در بر می‌گیرند:

```
com.easymeals.placeorder.menu
com.easymeals.placeorder.shoppingcart
com.easymeals.placeorder.customerdata
com.easymeals.placeorder.paymentdata
com.easymeals.placeorder.checkout
```

این نمونه نشان می‌دهد که هر ماژول در تک‌پارچهٔ ماژولار از یک یا چند مؤلفه تشکیل می‌شود (فصل ۸ را ببینید).

ماژولِ `PaymentProcessing` مسئولِ اعمالِ پرداخت است. `EasyMeals` کارت‌های اعتباری، کارت‌های نقدی و `PayPal` را می‌پذیرد؛ ماژولاریتیِ این معماری افزودنِ نوعِ پرداختِ دیگری (مانندِ امتیازهای وفاداری `(loyalty points)`) را آسان می‌کند. مشتریان این اطلاعات را در ماژولِ `PlaceOrder` وارد می‌کنند و این ماژول آن را به ماژولِ `PaymentProcessing` می‌فرستد. مؤلفه‌های این ماژول را می‌توان با فضاهای نامِ زیر بازنمایی کرد:

```
com.easymeals.payment.creditcard
com.easymeals.payment.debitcard
com.easymeals.payment.paypal
```

پس از پرداختِ سفارش، ماژولِ `PlaceOrder` با ماژولِ `PrepareOrder` ارتباط برقرار می‌کند و این ماژول کلِ سفارش را به کارکنانِ آشپزخانه نمایش می‌دهد. کارکنانِ آشپزخانه پس از پختن، سفارش را آمادهٔ ارسال علامت می‌زنند و سپس سفارش به ماژولِ `Delivery` فرستاده می‌شود. فضاهای نامِ زیر نمایانگرِ مؤلفه‌های درونِ ماژولِ `PrepareOrder` هستند:

```
com.easymeals.prepareorder.displayorder
com.easymeals.prepareorder.ready
```

ماژولِ `Delivery` یک پیک را به سفارش اختصاص می‌دهد و نشانیِ تحویل را مشخص می‌کند. این ماژول به پیک امکان می‌دهد سفارش را «تحویل‌شده» علامت بزند، که چرخهٔ عمرِ آن سفارشِ مشخص را به پایان می‌رساند، و هر مشکلی را (مانندِ یک سگِ پرخاشگر پشتِ درِ ورودی یا مشتری‌ای که در خانه نیست) ثبت کند. فضاهای نامِ زیر نمایانگرِ مؤلفه‌های درونِ ماژولِ `Delivery` هستند:

```
com.easymeals.delivery.assign
com.easymeals.delivery.issues
com.easymeals.delivery.complete
```

ماژولِ `Recipes` به آشپزها و کارکنانِ مدیریت امکان می‌دهد اقلامی به منو بیفزایند و فهرستِ مواد و مقادیرِ هر قلمِ منو را نگه‌داری کنند. فضاهای نامِ زیر نمایانگرِ مؤلفه‌های درونِ ماژولِ `Recipes` هستند:

```
com.easymeals.recipes.view
com.easymeals.recipes.maintenance
```

سرانجام، ماژولِ `IngredientsInventory` اطمینان می‌دهد که برای دستورهای پختِ منو موادِ اولیهٔ کافی در دسترس باشد. این ماژول اندکی پیچیده‌تر از ماژول‌های دیگر است: یک مؤلفهٔ هوش مصنوعیِ پیشرفته دارد که حجمِ فروش را پیش‌بینی می‌کند تا فرایندِ تهیهٔ موادِ اولیهٔ هفته را خودکار کند.

فضاهای نامِ زیر نمایانگرِ مؤلفه‌های درونِ ماژولِ `IngredientsInventory` هستند:

```
com.easymeals.inventory.maintenance
com.easymeals.inventory.forecasting
com.easymeals.inventory.ordering
com.easymeals.inventory.suppliers
com.easymeals.inventory.invoices
```

و تمام! سادگی و سطحِ ماژولاریتیِ تک‌پارچهٔ ماژولار، یافتن و نگه‌داریِ کد برای رفعِ یک اشکال یا افزودنِ یک قابلیتِ تازه را نسبتاً آسان می‌کند. این نمونه قدرتِ این سبک معماریِ ساده و سرراست را نشان می‌دهد.

واژه‌نامهٔ اختصاصی این فصل

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Modular Monolith	تک‌پارچهٔ ماژولار
Web Archive (WAR)	آرشیو وب
Assembly (.NET)	اسمبلی

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Enterprise Archive (EAR)	آرشیو سازمانی
Isomorphic Shape	شکل هم‌ریخت
Namespace	فضای نام
Module	ماژول
Monolithic Structure	ساختار تک‌پارچه
Modular Structure	ساختار ماژولار
Self-Contained Artifact	مصنوع خودبسنده
Peer-to-Peer	همتابه‌همتا
Compile-Time Dependency	وابستگی زمان کامپایل
Shared Interface Class	کلاس واسط مشترک
DLL Hell (antipattern)	جهنم DLL
JAR Hell (antipattern)	جهنم JAR
Mediator	میانجی
Abstraction Layer	لایهٔ انتزاعی
Monolithic Database Topology	توپولوژی پایگاه‌دادهٔ تک‌پارچه
Contextual Data	دادهٔ زمینه‌ای
On-Demand Provisioning	تأمین منابع بر حسب تقاضا
Unstructured Monolith	تک‌پارچهٔ بی‌ساختار
Intermodule Communication	ارتباط میان‌ماژولی
Package Structure	ساختار بسته‌ها
Automated Governance	حاکمیت خودکار
Pseudocode	شبه‌کد
Cross-Functional Team	تیم چندتخصصی

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Architectural Direction	جهت‌گیری معماری
Domain-Focused Team	تیم حوزه‌محور
Wishlist	فهرست علاقه‌مندی‌ها
Loyalty Points	امتیازهای وفاداری
Service-Based Architecture	معماری مبتنی بر سرویس
Microservices	میکروسرویس
Lifecycle	چرخهٔ عمر